

RobomationLAB-코딩-가이드

RobomationLAB 코딩 가이드

코딩-가이드

코딩 가이드

이 문서는 로보메이션에서 개발한 '로봇 동작 스트리밍 서비스를 위한 실행 엔진 및 통합 저작환경' 중 하나인 로봇 코딩 교육용 플랫폼, RobomationLAB(로보메이션 랩)에 대해 설명합니다.

다음과 같은 순서로 자세한 설명을 제공합니다:

- RobomationLAB의 주요 특징
- RobomationLAB 로봇 프로그래밍 방식
- RobomationLAB 로봇 코딩 기본 문법 체계

목차

. RobomationLAB 로봇 코딩 교육 플랫폼

-) 로봇 코딩 프로그램
-) 블록 파이썬 실시간 양방향 변환
-) 코딩 프로그램 주요 특징
-) 실시간 로봇 제어 방식

RobomationLAB 로봇 프로그래밍 방식

-) 순차 실행과 병렬 실행
-) setup 함수
-) loop 함수

RobomationLAB 로봇 코딩 기본 문법 체계

-) robomation 패키지 가져오기
-) 로봇 인스턴스 생성
-) 로봇 제어 메서드
-) 센서 및 상태 읽기
-) 이벤트 감지
-) 동작 완료 대기 (wait)
-) 유틸리티 함수 (Utils)

RobomationLAB 로봇 코딩 교육 플랫폼

RobomationLAB은 **AI 로보틱스 SW 교육**에 최적화된 플랫폼으로, 초등/중학생 대상 로봇 코딩 교육을 위한 Chrome 웹 브라우저 기반의 통합 저작환경을 제공합니다.

RobomationLAB은 **블록 코딩, 스크립트 코딩** 등 다양한 방식의 로봇 코딩 교육용 프로그램들을 제공합니다.

코딩의 이론을 학습할 뿐만 아니라 로보메이션의 제품들을 활용해 실제로 로봇을 움직이고 제어하면서, 코딩과 로봇을 동시에 학습할 수 있는 기회를 제공합니다.

로봇 코딩 프로그램

Block Composer (블록 컴포저)

Block Composer는 블록 코딩을 통해 자사 로봇들을 쉽고 빠르게 제어하며, 로봇 제어의 기초를 학습할 수 있는 도구입니다.

- 피지컬 컴퓨징에 최적화된 저작 환경
- 블록 Drag&Drop 방식으로 초보자도 쉽게 코딩 가능
- 기본 개념부터 문법 오류 없는 학습 환경 제공

- Python 스크립트 코드로 자동 변환
- 로봇 별로 미리 정해진 기능을 가진 블록 모음과 다양한 체험 예제 제공
- 코드 실행을 통해 실시간 결과 확인 가능
- 블록 조합으로 문제 해결 능력 및 창의력 향상
- AI 기반 스크립트 코드 분석을 통해 최적화된 피드백 제공

Script Composer (스크립트 컴포저)

Script Composer는 스크립트 코딩을 통해 자사 로봇들을 쉽고 빠르게 제어하며, Python, Javascript 문법 및 로봇 코딩의 기초를 학습할 수 있는 도구입니다.

- Python, Javascript 에디터 동시 제공
- 언어 별 코드 자동 완성 및 코드 삽입 기능 제공
- 로봇 별 다양한 체험 예제 코드 제공
- 코드 실행을 통해 실시간 결과 확인 가능
- AI 기반 스크립트 코드 분석을 통해 최적화된 피드백 제공

코딩 프로그램 주요 특징

RobomationLAB에서 제공하는 로봇 코딩 프로그램의 주요 특징은 다음과 같습니다.

- . 크롬 웹 브라우저 기반으로, OS(운영체제)의 제약을 받지 않음
- . Web Serial 통신 기반으로, USB 동글을 통해 로봇 하드웨어를 직접 제어
- . 멀티 로봇 동시 제어 지원 - 로봇의 종류, 수 제한 없음
- . 파일 저장 시 결과물은 JSON 텍스트 파일로 변환하여 저장

실시간 로봇 제어 방식

RobomationLAB에서 제공하는 로봇 코딩 프로그램에서는 다음과 같은 과정을 통해 실시간으로 로봇을 제어합니다.

- . 블록코딩 또는 스크립트 코딩을 통해, 로봇을 제어하기 위한 Effector, Command 객체의 값을 설정하거나 로봇의 Sensor 값 및 Event 발생을 활용하는 코드를 작성합니다.

- . 코드를 실행합니다.
- . Web Serial 통신을 통해 로봇으로부터 Sensor 및 Event 데이터를 담은 패킷을 받아, 로봇 Device 객체에 반영합니다.
- . 실시간으로 코드를 해석하여, Effector, Command 객체에 데이터를 덮어쓰거나, Sensor, Event 객체의 값을 읽어옵니다.
- . 로봇 Device 객체의 데이터를 반영한 패킷을 생성한 뒤, Web Serial 통신을 통해 로봇에 전송하여 실제로 로봇이 동작하는지 확인합니다.
- . 코드가 실행되는 동안, ~ 의 과정을 약 ~ ms 마다 반복 수행합니다.

블록 ↔ 파이썬 실시간 양방향 변환

RobomationLAB 은 블록 코딩 에디터와 파이썬 코딩 에디터를 함께 제공하며, 두 에디터는 실시간으로 양방향 동기화됩니다.

- 블록 코딩 에디터에서 블록을 추가·수정·삭제하면, 그 변경사항이 실시간으로 파이썬 코드에 반영됩니다.
- 반대로 파이썬 코딩 에디터에서 코드를 수정하면, 그 변경사항이 실시간으로 블록에 반영됩니다.

이처럼 블록과 파이썬 코드가 대 로 대응되어 양방향으로 변환되므로, 블록으로 전체 구조를 빠르게 만든 뒤 파이썬으로 세부 코드를 다듬거나, 블록과 비교하며 파이썬 문법을 학습하는 등 블록 코딩과 스크립트 코딩을 넘나드는 유기적인 코딩이 가능합니다.

단, 파이썬 → 블록 변환은 블록으로 표현할 수 있는 코드에 한해 이루어집니다. (잘못된 문법이거나 블록으로 변환할 수 없는 코드인 경우 전환이 되지 않을 수 있습니다. 자세한 내용은 코딩 규칙 문서를 참고하세요.)

RobomationLAB 로봇 프로그래밍 방식

순차 실행과 병렬 실행

로봇을 프로그래밍하는 방식에는, 순차 실행 방식과 병렬 실행 방식이 있습니다.

순차 실행은, 한 동작이 마무리된 뒤에 다음 동작을 수행하는 방식으로, 단순한 행동을 코딩하는 데에 적합합니다.

예를 들어, 로봇을 앞으로 이동한 이후에 멈춰서 LED를 켜고 싶다면, 각 동작에 해당하는 코드를 순서대로 배치하여 시간순으로 실행할 수 있도록 순차 실행 방식이 가능해야 합니다.

병렬 실행은, 여러 동작을 동시에 수행하는 방식으로, 보다 복잡하고 고도의 행동을 프로그래밍하는 데에 필요합니다.

예를 들어, 족보행 로봇이 걷는 동작을 구현하고자 한다면, 로봇의 발과 다리들을 동시에 움직여야 보행이 가능하기 때문에 병렬 실행 방식의 코딩이 가능해야 합니다.

RobomationLAB에서 제공하는 로봇 코딩 프로그램은,

Arduino의 H/W 개발 환경과 유사한 setup / loop 코드 구조를 기반으로, 순차 실행 방식과 병렬 실행 방식을 동시에 지원합니다.



Block Composer 에 처음 접속하면 다음과 같이 두 개의 빈 함수 블록이 작업공간에 표시되는데,

'시작하기' 블록은 `setup` 함수를, '무한 반복하기' 블록은 `loop` 함수를 나타냅니다.

블록은 실시간으로 파이썬 코드로 변환되며, 코드는 다음과 같은 기본 구조를 갖습니다.

```
# 파이썬 코드 기본 구조
from robomation import *

# put setup code here, to run once:
def setup():
    pass

# put control code here, to run repeatedly:
def loop():
    pass
```

로봇을 제어하려면 코드 최상단에 `from robomotion import *` 를 두고, 사용할 로봇을 인스턴스로 선언합니다. (예: `hamster_s = HamsterS(0)`)

setup 함수

setup 함수는 '코드 실행'을 하는 순간 단 한번만 수행됩니다.

setup 함수에서는 주로 변수를 초기화하거나 로봇의 모드, 기능 등을 초기화하는 코드를 작성합니다.

예를 들어, 바퀴를 통해 움직이는 로봇을 제어할 때, setup 함수에서는 바퀴의 초기 속도를 설정할 수 있습니다.

함수 내에서 시간 지연이 필요한 경우, `Utils.wait(...)` 함수를 사용하면 마치 동기 방식처럼 시간순으로 동작하는 코드를 작성할 수 있습니다.

`Utils.wait(...)` 는 지정한 시간(초)만큼 기다린 뒤 다음 코드를 수행하므로, 정해진 시간 또는 동작 이후에 깨어나 다음 코드를 이어서 실행합니다.

이 기능을 활용하면, 간단한 순차 실행 뿐 아니라 병렬 실행 역할을 하는 loop 함수와의 연동을 통해, 강력한 로봇 프로그래밍이 가능할 것입니다.

다음은 HamsterS 로봇이 초간 전진 후 초간 후진하는 코드를 작성하는 예시입니다.

병렬 실행 방식의 loop 함수 내에서 위 동작을 구현하려면, 시간 계산과 제어 코드가 혼합되어 코드가 매우 복잡해지게 됩니다.

대신, setup 함수 내에서 `Utils.wait` 시간 지연 함수를 사용하면, 마치 동기 방식처럼 시간순으로 동작하는 코드를 작성할 수 있습니다.

(`Utils.wait` 함수에 대해서는 이후 유틸리티 함수 (Utils) 에서 다시 설명합니다.)

예시 코드 (파이썬)

```
from robomotion import *

hamster_s = HamsterS(0)

# put setup code here, to run once:
def setup():
    # 양쪽 바퀴 속도를 50으로 설정하여 앞으로 이동
    hamster_s.set_wheel_speed('both', 50)
    Utils.wait(1) # 1초 기다리기
    # 양쪽 바퀴 속도를 -50으로 설정하여 뒤로 이동
    hamster_s.set_wheel_speed('both', -50)
    Utils.wait(1) # 1초 기다리기
    hamster_s.stop()
```

```
# put control code here, to run repeatedly:
def loop():
    pass
```

(로봇 인스턴스 및 메서드 문법에 대해서는 이후 RobomationLAB 로봇 코딩 기본 문법 체계 에서 다시 설명합니다.)

loop 함수

loop 함수는 병렬 실행을 지원하며, 코드가 실행되는 동안 약 ms 마다 반복하여 수행됩니다.

loop 함수에서는 주로 변수의 값을 반복해서 설정하거나 로봇의 특정 이벤트의 발생을 감지하여 처리하는 코드를 작성합니다.

다음은 시간에 따라 HamsterS 로봇의 바퀴 속도와 LED 색상이 변하는 코드를 작성하는 예시입니다.

(setup / loop 함수 밖에서 선언한 변수를 함수 안에서 변경할 때에는 `global` 키워드로 선언합니다.)

```
from robomation import *

hamster_s = HamsterS(0)
frame = 0

# put setup code here, to run once:
def setup():
    global frame
    frame = 0

# put control code here, to run repeatedly:
def loop():
    global frame
    frame += 1 # loop 함수가 호출될 때마다 frame 변수의 값 1씩 증가

    # 바뀐 frame 값을 활용하여, 양쪽 바퀴 속도와 양쪽 LED의 RGB 값 설정
    hamster_s.set_wheel_speed('both', frame % 100)
    hamster_s.set_led_color('left', frame % 256, 0, 0)
    hamster_s.set_led_color('right', 0, 0, frame % 256)
```

(로봇 인스턴스 및 메서드 문법에 대해서는 이후 RobomationLAB 로봇 코딩 기본 문법 체계 에서 다시 설명합니다.)

다음은 HamsterS 로봇의 몸체를 가볍게 두드리는 Tap 동작이 발생하면, LED를 적색으로 켜는 코드를 작성하는 예시입니다.

```
from robomation import *

hamster_s = HamsterS(0)

# put setup code here, to run once:
def setup():
    pass

# put control code here, to run repeatedly:
def loop():
    # Tap 동작이 발생하는 순간, 이벤트 발생 감지
    if hamster_s.tap():                # 이벤트 감지 시, True
        hamster_s.set_led_color('both', 'red') # 양쪽 LED를 적색으로 켜기
    else:
        hamster_s.turn_off('both')      # 양쪽 LED 끄기
```

(이벤트 감지(tap()) 등 문법에 대해서는 이후 RobomationLAB 로봇 코딩 기본 문법 체계 에서 다시 설명합니다.)

RobomationLAB 로봇 코딩 기본 문법 체계

RobomationLAB에서 제공하는 로봇 코딩 프로그램에서 코드를 작성할 때 지켜야 할 기본 문법 체계는 다음과 같습니다.

ver . . . 부터 로봇 제어는 robomation 파이썬 패키지를 통해 이루어지며, 로봇을 인스턴스로 만든 뒤 그 인스턴스의 메서드를 호출하는 방식으로 코드를 작성합니다.

robomation 패키지 가져오기

로봇 제어에 필요한 클래스(로봇)와 유틸리티(utils)를 사용하려면, 코드 최상단에 다음 줄을 포함합니다.

```
from robomation import *
```


로봇 인스턴스 생성

사용할 로봇을 인스턴스로 선언합니다.

클래스 이름은 로봇 종류를, 괄호 안의 숫자는 인스턴스 인덱스(부터 시작)를 나타냅니다.

```
hamster_s = HamsterS(0)   # 햄스터 S 1대
```

로봇별 클래스 이름과 기본 변수명은 다음과 같습니다.

로봇	클래스 이름	기본 변수명
햄스터 S	HamsterS	hamster_s
햄스터	Hamster	hamster
삐오	Pio	pio
터틀	Turtle	turtle
비글	Beagle	beagle
라쿤봇	RaccoonBot	raccoon
치즈스틱	CheeseStick	cheesestick

같은 종류의 로봇을 여러 대 사용할 경우, 인덱스를 , , ...로 늘려 선언합니다.

```
hamster_s = HamsterS(0)
hamster_s_1 = HamsterS(1)
```

로봇 제어 메서드

로봇을 제어할 때에는 인스턴스의 메서드를 호출합니다.

메서드는 바퀴 속도 설정, 이동, LED, 소리 등 로봇의 동작을 수행합니다.

```
hamster_s.set_wheel_speed('both', 50)   # 양쪽 바퀴 속도 설정
hamster_s.move_distance(10, 'cm')       # 10cm 앞으로 이동
hamster_s.set_led_color('both', 'red')   # 양쪽 LED를 적색으로
```

로봇별 사용 가능한 메서드 목록과 매개변수는 각 로봇 문서('햄스터-S' 등)를 참고하세요.

센서 및 상태 읽기

센서 값이나 로봇의 상태를 읽을 때에도 메서드를 호출하며, 메서드가 그 값을 반환합니다.

```
left = hamster_s.floor('left')      # 왼쪽 바닥 센서 값 읽기
dist = hamster_s.proximity('right')  # 오른쪽 근접 센서 값 읽기
```

이벤트 감지

상태 변화나 환경의 변화로 발생하는 이벤트는, 해당 이벤트 메서드가 이벤트가 발생한 순간에 True 를 반환하는 방식으로 감지합니다.

```
if hamster_s.tap():                  # 두드림(Tap) 이벤트가 발생한 순간 True
    hamster_s.set_led_color('both', 'red')
```

동작 완료 대기 (wait)

이동·회전·소리 재생처럼 완료까지 시간이 걸리는 메서드는 `wait` 매개변수를 갖습니다.

- `wait=True` (기본값): 동작이 완료될 때까지 기다린 뒤 다음 코드를 수행합니다.
- `wait=False`: 동작을 시작하고 곧바로 다음 코드를 이어서 수행합니다.

```
hamster_s.move_distance(10, 'cm', wait=True)  # 이동이 끝날 때까지 기다림
hamster_s.move_distance(10, 'cm', wait=False)  # 이동을 시작하고 바로 다음 코드 실행
```

행

유틸리티 함수 (Utils)

시간 지연, 소리 재생, 로그 출력, 색상 등 로봇 종류와 무관한 공통 기능은 `Utils` 클래스의 메서드를 통해 사용합니다.

```
Utils.wait(1)                        # 1초 기다리기
Utils.speak('안녕하세요')            # 텍스트 음성(TTS) 재생
Utils.log(0, '', '')                 # 콘솔 로그 출력
```

자세한 유틸리티 함수 목록은 [커스텀 함수 문서](#)를 참고하세요.

코딩-규칙

코딩 규칙

이 문서는 RobomationLAB 에서 로봇 코딩을 할 때 지켜야 할 코딩 규칙에 대해 설명합니다.

참고: 스크립트 코딩은 **Python 전용**으로 동작하며, 블록은 **robomation** 파이썬 패키지를 사용하는 Python 코드로 대 변환됩니다.

순서 안내: 공통 핵심 규칙(~) → 블록 코드 규칙(~) → 파이썬 스크립트 코드 규칙(~) 순으로 정리되어 있습니다. 블록 코드와 파이썬 코드 생성 모두 중요하게 지켜야 합니다.

1. 블록 ↔ 파이썬 1대1 변환 제약

블록 코딩 에디터와 파이썬 코딩 에디터는 대 로 대응되어 실시간으로 양방향 변환됩니다.

- 블록 → 파이썬: 블록은 항상 올바른 파이썬 코드로 변환됩니다.
- 파이썬 → 블록: **블록으로 표현할 수 있는 코드**에 한해 변환됩니다.

따라서 파이썬 에디터에서 다음과 같은 코드를 작성하면, 파이썬 → 블록 코딩 에디터로 전환이 되지 않을 수 있으며, 이 경우 전환이 실패한 원인이 함께 표시됩니다.

- . 파이썬 문법 오류가 있는 코드
- . 블록으로 대응되지 않는(변환 규칙에 정의되지 않은) 함수·메서드·문법을 사용한 코드

블록 코딩 에디터로의 전환을 정상적으로 유지하려면, 이 문서와 각 로봇 문서에 정의된 클래스·메서드·함수 및 지원되는 문법 범위 내에서 코드를 작성해야 합니다.

2. 코드 제시 규칙

robomation 파이썬 API 메뉴얼과 블록 **파이썬 변환 규칙 문서**에 명시된 클래스, 메서드 및 함수만 사용할 수 있습니다.

로봇 제어 시 위 문서에 정의되지 않은 새로운 로봇 제어 메서드를 생성하여 사용하는 것은 엄격히 금지됩니다. 단, 사용자가 새로운 함수의 생성을 요청한 경우에는 가능합니다.

위 문서에 명시된 변환 예시에 나타나지 않는 임시 변수(예: 로봇 인스턴스를 담는 변수 외의 추가 변수) 선언은 사용자의 변수 생성 요청 시에만 생성할 수 있습니다.

또한, 메서드의 매개값은 반드시 리터럴로 작성해야 합니다.

```
# 올바른 예시
hamster_s.set_wheel_speed('both', 100)

# 잘못된 예시 - API 메뉴얼에 명시되지 않은 함수 사용
hamster_s.set_wheel('both', 100)

# 잘못된 예시 - 임의 변수 선언
SPEED = 100
hamster_s.set_wheel_speed('both', SPEED)
```

3. 블록 코드 기본 구조

모든 블록 코드 제시 시, 프로그램의 진입점 역할을 하는 최상위 함수 블록인 시작하기와 무한 반복하기를 기본 구조로 항상 포함하여 제시합니다.

이 규칙을 추가하면, 앞으로 모든 블록 코드는 아래와 같은 기본 구조를 갖게 됩니다.

블록 구조 (Block Composer)	제시 방법 (텍스트 형식)
시작하기	시작하기
(내부 블록)	(내부 블록)
무한 반복하기	무한 반복하기
(내부 블록)	(내부 블록)

4. 블록 코드 형식 (줄 바꿈 및 들여쓰기 규칙)

- 최상위 블록(시작하기, 무한 반복하기 등)은 왼쪽 정렬합니다.
- 각 명령어 블록은 반드시 개행 문자를 사용하여 분리하여, 한 줄에 하나의 블록만 출력해야 합니다.

- 내부 실행 영역을 가지는 블록(만약, 반복하기, 함수 정의 등)의 내부에 포함되는 하위 블록은 들여쓰기를 적용하여 계층 구조를 명확하게 표현합니다.

5. 내부 블록 및 조건 표현 규칙

드롭다운 메뉴의 선택 값 또는 입력 값은 블록의 기능적 인자(Argument)에 해당하며, 블록의 문구 내에서 해당 값이 위치하는 곳에 대괄호([])를 사용하여 직접 삽입하여 표현합니다.

이는 블록의 고유 문구와 사용자가 선택/입력한 값을 통합하여 시각적으로 재현하는 역할을 합니다.

모든 블록 코드는 블록의 고유 명칭, 드롭다운 메뉴 선택 값, 그리고 사용자가 입력한 값을 대괄호([])를 사용하여 모두 포함하는 형태로, 실제 Block Composer의 블록 모양을 텍스트로 최대한 가깝게 재현하여 제시해야 합니다.

블록 구조 (Block Composer)	제시 방법 (텍스트 형식)'''
만약 [조건] 하기 [명령] 아니라면 [명령]	만약 [조건] 하기 [명령] 아니라면 [명령]
라쿤봇: [속도] 제어 모드로 설정하기	라쿤봇: [속도] 제어 모드로 설정하기
라쿤봇: 관절 [] 속도를 [] (으)로 정하기	라쿤봇: 관절 [] 속도를 [] (으)로 정하기

6. 스크립트 코드 기본 구조

모든 스크립트 코드(Python) 제시 시, 프로그램의 진입점 역할을 하는 함수인 setup()과 loop()를 기본 구조로 항상 포함하여 제시합니다.

또한, 로봇 제어에 필요한 클래스/유틸리티를 사용할 수 있도록 코드 최상단에 `from robomation import *`를 포함하고, 사용할 로봇은 인스턴스로 선언합니다.

이 규칙을 추가하면, 앞으로 모든 스크립트 코드(Python)는 아래와 같은 기본 구조를 갖게 됩니다.

```
# 파이썬 코드 기본 구조
from robomation import *

# (사용할 로봇이 있다면 인스턴스로 선언, 예시)
hamster_s = HamsterS(0)

# put setup code here, to run once:
def setup():
    pass
```

```
# put control code here, to run repeatedly:
def loop():
    pass
```

7. 스크립트 코드 형식 (줄 바꿈 및 들여쓰기 규칙)

- 최상위 함수(setup, loop 등)은 왼쪽 정렬합니다.
- 줄 바꿈 시에 발생하는 들여쓰기는 반드시 **탭(\t)** 개를 기준으로 합니다. (예 디터의 들여쓰기 기준과 동일)

8. 로봇 제어 규칙 (인스턴스 / 메서드)

로봇을 제어할 때에는 다음과 같은 규칙을 따라야 합니다.

- . 코드 최상단에 `from robomation import *` 를 포함합니다.
- . 사용할 로봇을 인스턴스로 선언합니다. 변수명은 로봇별로 정해진 기본 변수명을, 인덱스는 부터 부여합니다.
- . 선언한 인스턴스의 메서드를 호출하여 로봇을 제어합니다.

```
from robomation import *

hamster_s = HamsterS(0)

def setup():
    hamster_s.set_wheel_speed('both', 50)

def loop():
    pass
```

같은 종류의 로봇을 여러 대 사용하는 경우, 인덱스를 , , ... 로
늘려 선언합니다. (예: `hamster_s = HamsterS(0)` , `hamster_s_1 = HamsterS(1)`)

로봇의 바퀴 속도 설정·이동 등은 메서드 내부에서 필요한 초기화(예: 이전 이동 명령 정
리)를 자동으로 처리하므로, 별도의 선행 초기화 코드를 작성할 필요가 없습니다.

9. 유틸리티 함수 호출 규칙

시간 지연, 소리 재생, 로그 출력, 색상 등 공통 유틸리티 기능은 `Utils` 클래스의 메서드를 통해 호출합니다.

```
Utils.wait(1) # (seconds)
Utils.play_sound('', 100, False)
Utils.log(0, '', '')
```

자세한 유틸리티 함수 목록은 아래 문서를 참고하세요.

- 색상
- 소리
- 제어

10. 로봇 전용 메서드 우선 사용 규칙

로봇 하드웨어(바퀴 속도, LED, 소리 등) 제어 시,
공통 유틸리티 함수보다 로봇(예: '햄스터-S') 인스턴스의 전용 메서드를 우선해서 사용해야 합니다.

예를 들어, 로봇을 이용해 소리를 내야 하는 경우,
유틸리티 함수인 `Utils.play_sound()` 보다 '햄스터-S' 인스턴스의 `sound_clip()` 메서드를 우선해서 사용해야 합니다.

```
# 햄스터 S 로봇을 통해 소리를 냅니다. (권장)
hamster_s.sound_clip('beep')
```

```
# PC / 태블릿 / 모바일 디바이스를 통해 소리를 냅니다. (비권장)
Utils.play_sound('beep', 100, False)
```

11. Import 규칙

파이썬 코드 작성 시에는, 코드 실행에 필요한 모듈만 import 합니다.

- 로봇 제어를 위해 코드 최상단에 `from robomotion import *` 를 기본적으로 포함합니다.
- `math.` 을 사용하는 코드에는 `import math` 가, `random.` 을 사용하는 코드에는 `import random` 이 자동으로 추가됩니다.
- 그 외 불필요한 모듈은 import 하지 않습니다.